

The Ledger: Addendum A1 to v10.3

Locating Unit State: A Refinement of §7

R. Delloye

April 2026

Abstract

Section 7 of v10.3 introduced a unit state dictionary $\text{state}_t(u)$ and noted that, for some instruments, the dictionary must be keyed by the pair (w, u) rather than by u alone. The phrasing was correct but coarse: it treated the per-unit and per- (w, u) cases as a binary switch and left unspecified where shared mutable data (settlement prices, strategy weights, lifecycle stages) live. This addendum refines the treatment by resolving unit state into three distinct stores: an immutable, versioned, per-unit *ProductTerms*; a mutable, per-unit, holder-shared *UnitStatus*; and a per-holder, per-unit *PositionState*. The refinement is not additive only: it removes the need for a separate wallet-keyed state store, because every economic fact that looked per-wallet is naturally keyed by (w, u) once mandates and strategies are admitted as a new sub-class of unit (contract units). Eleven conditions and four auxiliary definitions govern the use and mutation of these three stores. Under these, the negation of v10.3 Invariant P1 (conservation) and of v10.3 Invariant P8 (snapshot consistency, time travel) become structurally unreachable in the persistent ledger state; three additional invariants internal to this addendum (Q1–Q3: terms immutability, identity permanence, handler-field canon) are also structurally unreachable. The refinement supersedes the phrasing at v10.3 line 1034 and the accessor signature at line 2287; the rest of the main document is unchanged.

Contents

1	What This Addendum Changes, and What It Preserves	3
1.1	The Passage Under Refinement	3
1.2	What Is Preserved	3
1.3	Collapse of the Wallet Sector: What the Reader Needs to Know	3
2	Definitions	4
2.1	The Three Stores	4
2.2	Registered Units and Totality	4
2.3	Asset Units and Contract Units	4
2.4	Accessor and Carrier Disciplines	5
2.5	Events, Deltas, Conservation, and Replay	6
2.6	Fungibility and Amendments	7
2.7	Placing a New Field: Decision Tree	7
2.8	Compact Notation Used in This Addendum	7
3	The Three-Store Decomposition	9
3.1	What Lives Where	9
3.2	Why ProductTerms Is Versioned and Append-Only	9
3.3	Why UnitStatus Is Separate from ProductTerms	9
3.4	Why PositionState Is Per- (w, u) and Not Per- u	10
3.5	Two Orthogonal Disciplines on PositionState	10

4	The Eleven Conditions	11
4.1	The Wallet Registry Is Metadata, Not State	13
4.2	On Authorisation	13
5	How v10.3 §7 Is Refined	14
5.1	Replacement Text for Line 1034	14
5.2	Replacement Signature for Line 2287	14
5.3	What Else Moves	14
6	Four Test Cases	15
6.1	A Future Whose State Depends on Past Transactions	15
6.2	A Managed Account: The Inherently Wallet-Attached Case	15
6.3	A Quantitative Investment Strategy That Trades Futures	16
6.4	An Instrument Listed in the Universe but Not Yet Traded	18
7	Alternatives Considered	19
7.1	Two Stores (ProductTerms and PositionState Only)	19
7.2	Four Stores (Keeping an Explicit WalletState)	19
7.3	A Sentinel-Unit Scheme (“Dirac”)	19
7.4	Why Exactly Three	19
8	Structurally Unreachable Invariants	21
8.1	v10.3 Invariant P1: Conservation	21
8.2	v10.3 Invariant P8: Snapshot Consistency	21
8.3	Addendum Invariant Q1: Terms Immutability	22
8.4	Addendum Invariant Q2: Identity Permanence	22
8.5	Addendum Invariant Q3: Handler-Field Canon	22
8.6	What the Addendum Does Not Make Structurally Unreachable	22
9	Testing Targets	23
10	Risk Register for Migration	24
11	Reference Implementation	26
12	Summary	28

1 What This Addendum Changes, and What It Preserves

1.1 The Passage Under Refinement

Section 7.3 of v10.3 (line 1034) establishes unit state as an explicit object:

“Each tradable object has a unit identifier u . Beyond static fields (currency, maturity, notional), the framework associates an explicit state dictionary $\text{state}_t(u)$ at time t . Unit state is per-unit for most instrument types. For instruments with per-wallet state (e.g., futures accumulated cost), the state dictionary is per (wallet, unit) pair.”

The read accessor is introduced at line 2287 as `view.get_unit_state(unit)`: a function from u to the current state dictionary.

This phrasing mixes three distinct concerns into one dictionary: (i) static contractual terms that never mutate, (ii) per-contract observables—settlement prices, lifecycle stage, strategy weights—that are mutable but shared by all holders, and (iii) per-position accounting state—accumulated cost, high-water marks, accrued fees—that must differ between two wallets holding the “same” contract. The distinction between (ii) and (iii) is real, not cosmetic: (ii) is read by every holder but written by one canonical lifecycle handler; (iii) is read and written only under the identity of a particular (w, u) pair. Folding all three into `get_unit_state(u)` forces the implementation either to duplicate shared observables across holders (breaking conservation of observables) or to carve per- (w, u) data out of a per- u dictionary by convention (defeating the type system).

1.2 What Is Preserved

The three-store refinement is compatible with every guarantee v10.3 established. Conservation (Invariant P1) still holds for every unit by construction. Snapshot consistency / time travel (Invariant P8) still holds because the three stores are each a deterministic function of the event log. Referential integrity, atomic commitment, lifecycle purity, and PnL path-independence all survive unchanged. What changes is the internal shape of the state dictionary and the read signatures; the external guarantees do not. A reader who only wants the guarantees and not the internal shape may stop here.

The refinement introduces one substantive change to v10.3: it partitions the set $\mathcal{U}$ of units into *asset units* and *contract units*. Contract units (mandates, CSAs, strategy subscriptions) obey a restricted version of v10.3’s unit laws; Section 2.3 states the carve-outs and justifies them.

1.3 Collapse of the Wallet Sector: What the Reader Needs to Know

Section 6 of v10.3 introduced managed accounts and CSA margin agreements (Credit Support Annex, the ISDA collateral agreement attached to an OTC contract) as “wallet-level smart contracts”. This phrasing is compatible with two internal representations. The first gives each wallet an overlay dictionary containing per-mandate entries. The second promotes each mandate to a full unit, held by the client and issued by the manager, so that what looked like overlay fields are just *PositionState* rows. The second is adopted here, subject to the contract-unit carve-outs of Section 2.3. It respects conservation (by bookkeeping discipline, not by market mechanism), it composes correctly under multiple concurrent mandates on the same wallet, and it does not introduce a state sector whose keying convention differs from the rest of the ledger.

2 Definitions

This section defines every technical term the rest of the addendum uses. The style follows v10.3 §2: each term is stated as a definition and followed by a short gloss.

2.1 The Three Stores

Definition 2.1 (ProductTerms). *The ProductTerms store is a function*

$$\text{ProductTerms} : \mathcal{U}_t^{\text{reg}} \rightarrow \text{NonEmptyList}[\text{TermsVersion}],$$

total on the set $\mathcal{U}_t^{\text{reg}}$ of units registered at time t . It assigns to every registered unit a non-empty, append-only, chronologically ordered list of TermsVersion records. A TermsVersion is a snapshot of the contractual terms of the unit as of a given effective date. Existing versions are never mutated in place; amendments append a new TermsVersion. The current terms of a unit are the last element of its list.

Definition 2.2 (UnitStatus). *The UnitStatus store is a function*

$$\text{UnitStatus} : \mathcal{U}_t^{\text{reg}} \rightarrow \text{UnitStatusRecord},$$

total on the set of registered units at time t . It assigns to every registered unit a mutable status record shared across all holders. Fields of a UnitStatusRecord include the lifecycle stage (LISTED, ACTIVE, EXPIRED, SETTLED, CLOSED), the last published settlement price and its date, the current strategy weights (for a Quantitative Investment Strategy unit), the triggered-barrier flag (for barrier products), and the supersession link (Condition 4.7). UnitStatus is mutated by lifecycle handlers under the constraint that each field has exactly one canonical writer (Condition 4.10).

Definition 2.3 (PositionState). *The PositionState store is a partial function*

$$\text{PositionState} : \mathcal{W} \times \mathcal{U}_t^{\text{reg}} \rightharpoonup \text{PositionStateRecord},$$

assigning to a pair (a holder wallet and a unit) the per-position accounting state of that holder's position in that unit. Fields of a PositionStateRecord include the accumulated cost (for futures-like instruments), the clearinghouse binding when multiple clearinghouses are possible, the per-position OTC lifecycle cursor, the entry net asset value (for a subscription to a strategy), the high-water mark, accrued management and performance fees, the benchmark reference level at this holder's inception, and any mandate-breach flags. PositionState is partial: not every holder/unit pair has a row.

2.2 Registered Units and Totality

Definition 2.4 (Registered unit set). *The set $\mathcal{U}_t^{\text{reg}} \subseteq \mathcal{U}$ of units registered at time t is the set of unit identifiers written into the Unit Store by the **register** operation defined in Conditions 4.4 and 4.6. Registration atomically adds a unit to $\mathcal{U}_t^{\text{reg}}$, writes the initial TermsVersion, and writes the default UnitStatus. Registration is monotone: there is no **unregister** operator. Accessing an unregistered unit raises; accessing a registered unit always succeeds for ProductTerms and UnitStatus, and returns an optional value for PositionState.*

2.3 Asset Units and Contract Units

Definition 2.5 (Asset unit, contract unit). *The unit universe $\mathcal{U}$ partitions into two sub-classes.*

- An asset unit represents a tradable economic claim: cash in a given currency, a listed equity, a listed derivative on a specific CCP, an OTC trade, a bond, a structured note, a tokenised security, a managed-strategy share class. Asset units are the subject of v10.3 §3.2’s unit identity principle without modification. They are fungible-across-holders within their identity class, transferable between wallets through moves, divisible per their **multiplier** and currency conventions, and priceable under the standard valuation functional $V = \sum_u w(u)P(u)$.
- A contract unit represents a bilateral service agreement: a mandate, a CSA, a subscription agreement, a prime brokerage overlay. A contract unit has one issuer (the service provider) and one holder (the client), both named in its *ProductTerms*. It is not fungible across holders: two clients’ mandates differ in fee schedule, HWM methodology, and benchmark, and must not net against each other. It is not transferable in the move sense: a mandate is a relationship, not a bearer instrument, and it moves only by novation (a breaking amendment). It is not priceable under the standard valuation functional: it has no market price, only a residual accounting value expressed through the per-position fields.

Remark 2.6 (Carve-outs of v10.3 unit laws for contract units). *For a contract unit, the following v10.3 unit-level laws are modified or withdrawn.*

- Fungibility across holders (v10.3 §3.2): *withdrawn. Contract units are fungible only within a single holder’s position and only within a single TermsVersion.*
- Transferability via moves (v10.3 §2.3): *restricted to novation events, which are breaking amendments (Condition 4.7).*
- Valuation under $V = \sum_u w(u)P(u)$ (v10.3 §4): *withdrawn. Contract units contribute zero to portfolio valuation by the standard functional; the economic value of a contract manifests through the PositionState fields (accrued fees, HWM, mark of the underlying strategy) which are separately accounted.*
- Conservation (v10.3 §11 P1): *retained but by bookkeeping, not by market. Section 2.7 explains the distinction. The balance $\sum_w w(u) = 0$ holds trivially by the ± 1 issuance discipline: the manager issues with balance -1 , the client holds with balance $+1$. This is conservation as accounting discipline, not as market mechanism.*

Remark 2.7 (Accountability, not arithmetic). *The -1 balance the manager carries on a mandate is not an economic short position in the sense an OTC equity option short is. The manager does not owe the client a mandate back; the manager owes the client performance under the mandate. The Dirac sentinel-unit scheme considered in Section 7 fails precisely because no identifiable party stands behind the -1 side: the conservation sum is declared by fiat. The contract-unit scheme passes because the -1 is assigned to the manager, who is an accountable legal person. The distinction between the two schemes is one of accountability, not of balance arithmetic: both produce $\sum_w w(u) = 0$; only one has a named party standing behind it.*

2.4 Accessor and Carrier Disciplines

Definition 2.8 (Option accessor discipline). *The read accessor for PositionState returns an optional value:*

`position_state(w, u) : Option[PositionStateRecord].`

*The two return shapes are semantically distinct. A result of **None** means this wallet has never held this unit. A result of **Some(r)**, including the case where every field of *r* is zero, means this wallet has held this unit at some point and currently stands flat. Consumers that must distinguish “never held” from “held-and-closed”—variation margin settlement, wash-sale lookback for tax reporting, record-date entitlement calculations for dividends—depend on this distinction. Collapsing **None** into **Some** of a zero record is forbidden.*

Definition 2.9 (Growing key-set discipline). *The key set of PositionState is extensive: once a key pair has received a write, the key is never removed. Closing out a position leaves a row with*

every balance-like field equal to zero. The row is preserved for the lifetime of the unit. (This discipline was called “monotone carrier” in a preceding draft; the present name avoids collision with the order-theoretic sense of monotone.) The extensive property is what makes the replay operator a literal fold over the event stream—see Remark 2.16.

2.5 Events, Deltas, Conservation, and Replay

Definition 2.10 (StateDelta). A StateDelta is the output of an event handler: a structured record describing a proposed modification to one or more of the three stores. A StateDelta has three components: a list of ProductTerms appends, a list of UnitStatus field writes, and a list of PositionState row writes. A StateDelta is a mathematical object with no side effects; applying it is a separate step (Condition 4.3).

Definition 2.11 (Conserved field set). Each PositionState field has a conservation status declared at unit registration in `ProductTerms[u].field_spec`. The conservation status is immutable under preserving amendments and re-declared only on breaking amendments. For all units considered in this addendum, `accumulated_cost` and `balance` are conserved; `hum`, `accrued_fee`, `entry_nav`, and `mandate_breach_flags` are non-conserved. Write

$$\mathcal{F}_{\text{cons}}(u) = \{f : \text{field_spec}[u][f].\text{conserved} = \text{True}\}.$$

Definition 2.12 (Handler-level conservation). For each unit u touched by a handler’s output and each field $f \in \mathcal{F}_{\text{cons}}(u)$, the StateDelta Δ produced by the handler satisfies

$$\sum_{(w,u') \in \text{rows}(\Delta), u'=u} \Delta f(w,u) = 0.$$

The sum is over rows written by that single event, not over the full key space of PositionState; it is a per-event structural law, not a global audit. Induction over the event stream then yields the global invariant $\sum_w f(w,u) = 0$ (Theorem 8.1).

Definition 2.13 (Vacuous base case). A lifecycle event fires vacuously on a unit when the set of holders of that unit is empty at event time. For such an event the handler-level conservation constraint is discharged trivially because the sum is empty. The base case matters because it catches a common bug class—per-share division or per-holder fan-out computed without checking that holders exist—before it can occur. See Section 6.4.

Definition 2.14 (Apply and replay fold). Write `apply` for the deterministic function that applies a single event to a state snapshot, producing a new snapshot. Write `apply_from` for its iterated form over a list of events starting from a given snapshot:

$$\text{apply_from}(s, []) = s, \quad \text{apply_from}(s, e :: \text{rest}) = \text{apply_from}(\text{apply}(s, e), \text{rest}),$$

and let s_0 denote the empty state. The global replay fold is $\text{apply_all}(E) = \text{apply_from}(s_0, E)$.

Definition 2.15 (Structural unreachability (persistent)). A state predicate φ is structurally unreachable in the persistent ledger state when no sequence of well-typed events, applied through the `apply` boundary with conservation enforcement, can yield a committed state satisfying φ . The delta that would produce φ may still be constructible in handler code; conservation rejection at `apply` prevents it from reaching persistence. A stronger sense—“unreachable at any point in the handler computation”—would require refinement types on handler output and is a future direction; it is not claimed here. Throughout this addendum, “structurally unreachable” means unreachable in the persistent state, per this definition.

Remark 2.16 (Fold identity under extensive key-sets). *If the `PositionState` key set is extensive (Definition 2.9), then for every index k ,*

$$\text{apply_all}(E) = \text{apply_from}(\text{apply_all}(E[0:k]), E[k:]).$$

The equation is type-correct: both sides are states; the right-hand side threads an intermediate state through the split. The identity holds because no event’s behaviour depends on the absence of a key. If keys could be removed, a later event might observe a different key set depending on whether an earlier close-out preceded or followed the replay boundary, and the fold would no longer be associative.

2.6 Fungibility and Amendments

Definition 2.17 (Fungibility predicate). *The fungibility predicate is a product-declared function*
 $\text{is_fungibility_preserving} : \text{TermsVersion} \times \text{TermsAmendment} \rightarrow \{\text{PRESERVING}, \text{BREAKING}\}.$

It takes the current `TermsVersion` and a proposed amendment and returns whether the amendment preserves fungibility of existing positions with positions opened under the amended terms. The predicate must satisfy three properties to be accepted at registration:

- *Deterministic: the same (`TermsVersion`, `TermsAmendment`) input produces the same decision on every invocation.*
- *Monotone under field change: an amendment that changes a superset of the fields an already-classified-breaking amendment changes is itself breaking.*
- *Conservation-safe: any amendment that changes a field in the conservation-affecting dependency set—the settlement currency, the multiplier, or any field that defines membership in $\mathcal{F}_{\text{cons}}$ —is breaking.*

Predicates that violate any of the three are rejected at registration. A coupon-rate tweak within a declared corridor is typically preserving; an ISIN change, a benchmark substitution, a bond restructuring, or a currency change is breaking. The predicate itself is versioned and governed.

2.7 Placing a New Field: Decision Tree

Given a candidate field, the following procedure locates its store.

1. *Does the field’s value differ between two wallets holding the same unit?* If yes, it is per-holder: `PositionState[w, u]`.
2. *Does the field mutate after the unit is created?* If no, it is immutable: `ProductTerms[u]`.
3. *Otherwise*, it is mutable but shared: `UnitStatus[u]`.
4. *If the field represents a per-client fact about a mandate or strategy*, find the contract unit (mandate, CSA, subscription) and place the field on `PositionState[w, u]` under that contract unit—never on a per-wallet scalar.

Steps 1 and 2 together decide among the three stores. Step 4 handles the wallet-sector collapse: what looks like a per-wallet fact is really a per-position fact under an appropriate contract unit.

2.8 Compact Notation Used in This Addendum

In the prose, we refer to “the mandate unit”, “the strategy unit”, “the manager’s wallet”, and “the client’s wallet” rather than using subscripted symbols. Subscripts appear in equations only where disambiguation is essential—for the QIS example of Section 6.3, which simultaneously involves the strategy unit, the futures underlyings, a potential mandate wrapper, the strategy’s trading wallet, and the client’s wallet, we retain u_{QIS} , u_{ES} , w_{QIS} , w_C for readability.

The acronyms used in this addendum are: CCP (central counterparty, the clearinghouse into which exchange-traded derivatives are novated); CSA (Credit Support Annex, the ISDA collateral agreement attached to an OTC trade); CDM (Common Domain Model, the ISDA/FINOS canonical data model); HWM (high-water mark, the running maximum of a client's net asset value used to determine performance fees); NAV (net asset value); QIS (Quantitative Investment Strategy, a rule-based index-like product); SFTR (Securities Financing Transactions Regulation, EU); EMIR (European Market Infrastructure Regulation); UTI (Unique Transaction Identifier); LEI (Legal Entity Identifier); VM (variation margin, daily mark-to-market cash flow on a cleared derivative).

3 The Three-Store Decomposition

3.1 What Lives Where

Each of the three stores has a distinct mutation discipline, a distinct key, and a distinct canonical reader. Table 1 summarises the assignment.

Table 1: The three state stores. Each store is keyed differently and mutated by a disjoint set of handlers.

Store	Key	Representative fields
<i>ProductTerms</i>	u (immutable, versioned)	multiplier, currency, expiry, clearinghouse, strike, ISIN, fee schedule, mandate text, benchmark identity, index methodology, field conservation spec
<i>UnitStatus</i>	u (mutable, shared)	lifecycle stage, last settlement price, last settlement date, current strategy weights, NAV index level, triggered-barrier flag, supersession link
<i>PositionState</i>	(w, u) (partial)	accumulated cost, clearinghouse binding, per-position OTC lifecycle cursor, entry NAV, HWM, accrued management fee, accrued performance fee, benchmark reference level at inception, mandate breach flags

The remainder of this section justifies the three assignments. Section 4 then states the eleven conditions that govern their use.

3.2 Why ProductTerms Is Versioned and Append-Only

Unit terms change over time: a bond coupon steps up, a CSA’s eligible-collateral list is amended, a fee schedule is re-negotiated. An implementation that mutated the terms in place would lose the historical version, which is needed for three purposes: (a) regulatory reconstruction—the terms in force at a past settlement must be recoverable; (b) audit of past PnL decomposition, which is computed against the then-current terms; and (c) safe amendment rollback if a version is retracted before its effective date.

The append-only, non-empty list discipline serves these purposes at no cost to the common case: the current terms are the last element of the list and can be cached. The non-emptiness is a type-level guarantee that *ProductTerms* is never absent for a registered unit: registration writes the first *TermsVersion* atomically with the creation of the entry (Condition 4.6).

3.3 Why UnitStatus Is Separate from ProductTerms

ProductTerms and *UnitStatus* are both keyed by u , so one might ask whether they should be a single store. They should not, for two reasons.

First, the mutation disciplines differ. *ProductTerms* is append-only and written by a governance process (contract amendment). *UnitStatus* is written on every settlement, every corporate action, every strategy rebalance, and every lifecycle stage transition. Merging them would force one of two compromises: either *UnitStatus* inherits append-only semantics, producing a new *TermsVersion* on every mark-to-market—a write amplification of orders of magnitude—or *ProductTerms* inherits in-place mutation, defeating the amendment audit trail.

Second, the readers differ. *ProductTerms* is read by pricing and by amendment processing. *UnitStatus* is read by every holder during variation margin settlement and by every client looking

up current strategy weights. Co-locating data with different read patterns behind a single accessor hides the difference from the caller and makes caching decisions harder. Separation makes the access pattern explicit.

3.4 Why PositionState Is Per- (w, u) and Not Per- u

Two wallets holding the “same” contract can have different accounting state even when the contract is fully fungible as an asset unit. A manager and a client both long the same E-mini S&P contract have different entry prices and therefore different accumulated cost; a client subscribed to a strategy in March and a client subscribed in July have different high-water marks even though both are long the identical strategy unit; two clients whose mandates bind on different dates have different benchmark reference levels.

There is no per- u data layout that can encode distinct accounting state for two simultaneous holders without either (a) duplicating the row under a synthetic key that includes the holder, in which case the store is in fact per- (w, u) with extra ceremony, or (b) aggregating across holders and losing the distinction. Keying PositionState by (w, u) is the only layout that preserves holder distinguishability by types rather than by convention.

3.5 Two Orthogonal Disciplines on PositionState

PositionState is governed by two disciplines that operate on different axes and are both required.

The first is the Option accessor discipline of Definition 2.8: reading a key that has never been touched returns **None**, and reading a key that is currently flat but has been touched returns **Some** of a zero record. This is a discipline on the *type* of the read result.

The second is the growing key-set discipline of Definition 2.9: once a key has been written, it is never deleted. This is a discipline on the *storage* layout.

The two are orthogonal. The Option accessor could in principle be provided by a store that deletes closed-out rows (returning **None** instead of **Some** of zero), and an extensive store could in principle expose a bare accessor that defaults missing keys to a zero record (collapsing **None** and **Some** of zero). Neither of these is acceptable for the framework. A deleting store would break Remark 2.16: replay would cease to be associative because the key set would depend on whether close-outs preceded or followed the replay boundary. A collapsing accessor would break the semantic distinction on which wash-sale reporting and record-date entitlements depend.

Both disciplines are therefore required, and they are required independently. This is the first of the eleven conditions.

4 The Eleven Conditions

The three-store decomposition is necessary but not sufficient. Eleven additional conditions govern how the stores are written, read, and registered. They fall into four groups:

- *Structural* (C1, C3, C5, C9, C11): govern the shape of the stores.
- *Write-discipline* (C2, C10): govern who may mutate what, preserving what.
- *Registration-totality* (C4, C6, C8): govern what must exist at registration and what to do with the empty case.
- *Amendment* (C7): governs the two-track fungibility rule.

A fifth group, *authorisation*, is deferred: cross-wallet read authorisation is governed by a separate capability layer outside the scope of this addendum.

Each condition is stated as a property, explained by the failure mode it prevents, and referenced to the layer that enforces it (types, schema, or handler obligation) with a specific pointer to the reference implementation of Section 11.

Condition 4.1 (Option accessor and extensive carrier, both). *The `PositionState` accessor returns `Optional[PositionStateRecord]` (Definition 2.8) and the key set is extensive (Definition 2.9).*

Failure mode prevented. *A bare (non-Option) accessor collapses “never held” into “held and flat”, silently producing false-positive results for wash-sale lookback, record-date entitlement, and historical holdings queries. A non-extensive store breaks replay associativity (Remark 2.16) and forces lifecycle handlers to distinguish “closed on replay path A” from “closed on replay path B”.*

Enforcement. *Two layers. (1) The application API of `Ledger.position` (see §11) returns `Optional[PositionState]` and exposes no delete operator. (2) The persistence schema revokes row-deletion privileges from the application database role. Under these two, no code path in the application can construct the illegal state. Admin-level access is outside this threat model and governed by separate controls.*

Condition 4.2 (Handler-level conservation with vacuous base case). *Each event handler produces a `StateDelta` whose sum over rows of each conserved field is zero (Definition 2.12). The case of zero holders is included as a vacuous base case (Definition 2.13).*

Failure mode prevented. *A handler that distributes a cash flow per holder but fails to handle the zero-holder case may divide by zero, or more insidiously, may silently emit no moves while updating `UnitStatus`—leaving the ledger apparently consistent but semantically broken. The vacuous case is also the simplest unit test.*

Enforcement. *By handler obligation: `Ledger.apply` (see §11) sums the delta’s row values per conserved field at the `apply` boundary and raises on non-zero sum; no uncovered delta reaches persistence. Contrast C3 below.*

Condition 4.3 (Atomic `StateDelta` across the three stores). *A `StateDelta` is applied atomically across `ProductTerms`, `UnitStatus`, and `PositionState`. Partial application is rejected.*

Failure mode prevented. *A rebalance event that updates `UnitStatus` (new strategy weights) but fails to update the strategy’s `PositionState` rows (corresponding futures trades) leaves the reported strategy weights inconsistent with the strategy’s actual holdings—a silent divergence that no downstream consumer can detect without cross-store reconciliation.*

Enforcement. *By the storage layer: the three-store write in `Ledger.apply` is wrapped in a single database transaction at the persistence boundary.*

Relationship to C2. *C2 is a property of the `StateDelta` qua mathematical object: the sum over rows of each conserved field is zero, enforceable at handler output. C3 is a persistence discipline: the delta is applied as a single atomic write across the three stores. C2 governs the delta’s content; C3 governs the write’s atomicity. Both are needed: C2 without C3 allows torn writes; C3 without C2 persists non-conserving state atomically.*

Condition 4.4 (UnitStatus total at registration). *For every unit registered in $\mathcal{U}_t^{\text{reg}}$, UnitStatus is present and fully initialised with product-declared default values at registration time.*

Failure mode prevented. *A handler that reads UnitStatus before any trade has touched the unit—for example, on listing-day lifecycle activation—must not observe an absent entry. Registration totality makes every lifecycle transition from LISTED to ACTIVE to EXPIRED expressible purely through UnitStatus updates, with no PositionState row required.*

Enforcement. *By schema: `Ledger.register` (see §11) atomically writes the first UnitStatus record alongside the first ProductTerms version.*

Condition 4.5 (ProductTerms versioned and append-only). *ProductTerms of a unit is a `NonEmptyList[TermsVersion]` written only by appending. In-place mutation of an existing TermsVersion is forbidden.*

Failure mode prevented. *A mutable TermsVersion destroys the audit of amendment history and makes past PnL non-reconstructable (past moves were made against the then-current terms, which would no longer exist). The type also forestalls an absent-terms bug: a non-empty list has a first element by construction.*

Enforcement. *By types: the `NonEmpty` type in §11 has an append-only API and no mutation method.*

Condition 4.6 (ProductTerms total at registration). *For every unit in $\mathcal{U}_t^{\text{reg}}$, ProductTerms contains at least one TermsVersion, written atomically at registration.*

Failure mode prevented. *Separating registration of a unit from the first terms write allows a window in which the unit is referenced but has no defined contractual terms, making every downstream computation on it ill-defined.*

Enforcement. *By schema: `Ledger.register` (see §11) takes a TermsVersion as a mandatory argument.*

Condition 4.7 (Amendment is two-track, governed by the fungibility predicate). *An amendment to the terms of a unit is classified by the product-declared fungibility predicate (Definition 2.17) as either preserving or breaking. A preserving amendment appends a new TermsVersion to the unit’s ProductTerms list and leaves existing PositionState rows untouched. A breaking amendment allocates a fresh unit identifier, writes the new terms as that unit’s ProductTerms, stamps a supersession link in the old unit’s UnitStatus, and emits an atomic re-subscription StateDelta that re-keys positions from the old unit to the new while preserving the conservation sum.*

Failure mode prevented. *Mutating a unit’s terms in place when the amendment is breaking silently changes the economic content of every outstanding position. Allocating a fresh unit for every amendment—including those that preserve fungibility—bloats the unit universe and destroys netting.*

Enforcement. *By handler obligation: `Ledger.amend` (see §11) branches on the fungibility predicate’s output and invokes the preserving or breaking path accordingly.*

Condition 4.8 (Handlers discharge conservation vacuously on zero-holder units). *Each handler’s conservation obligation includes the empty-rows case (Definition 2.13).*

Failure mode prevented. *A dividend handler implemented as `per_share = total / n_holders` fails with division by zero on a listed-but-untraded security; more subtly, a rebalance handler that reads the set of holders and distributes pro-rata can silently update UnitStatus without emitting any PositionState writes when no one holds the unit. Both are caught by the empty-rows obligation.*

Enforcement. *By handler obligation and by test: the zero-holder case is a mandatory unit test in the handler test suite. A handler that fails to return explicitly on the empty case cannot be registered through the handler-registration path of §11.*

Condition 4.9 (Re-registration is a hard error). *Attempting to register a unit whose identifier already exists in $\mathcal{U}_t^{\text{reg}}$ is rejected. Registration is never a silent reset.*

Failure mode prevented. *A silent reset would destroy the amendment history, the lifecycle stage, and all existing PositionState rows under the old identity, while leaving outstanding references—in cached trade confirmations, in pending settlement instructions, in external regulatory reports—dangling.*

Enforcement. *By schema: Ledger.register performs a compare-and-set against the registry and raises on collision.*

Condition 4.10 (Each PositionState field is tagged with its canonical handler). *Every field of PositionStateRecord is annotated with the unique handler class that is permitted to mutate it: accumulated_cost by settlement and trade handlers, hum by the fee-crystallisation handler, entry_nav by the subscription handler, accrued_management_fee by the fee-accrual handler, and so on. Mutation by any other handler is a type error.*

Failure mode prevented. *When multiple handlers can write the same field, idempotency becomes a cross-handler coordination problem: replaying an event produces different results depending on which handler wrote a given field last. Field-canon tagging makes idempotency a per-key deduplication within a single handler class.*

Enforcement. *By types: field access is gated by handler-class tokens in the FIELD_SPEC of §11.*

Condition 4.11 (No per-wallet scalar store for economic state). *All per-holder economic state with respect to a mandate or strategy lives at PositionState under the appropriate contract unit. Flat per-wallet scalars holding economic state—a wallet-keyed HWM, a wallet-keyed accrued fee—are not permitted.*

Failure mode prevented. *A flat per-wallet scalar collapses the mandate dimension: a client holding two mandates on the same wallet (base and overlay, or two concurrent strategies) cannot carry two HWMs, two accrued fee balances, two breach flags. The single per-wallet scalar would silently aggregate or overwrite.*

Enforcement. *By schema: the WalletRegistry (§4.1) declares only metadata fields and no economic scalars.*

4.1 The Wallet Registry Is Metadata, Not State

There is a fourth store in the system, but it holds no economic state:

Definition 4.12 (WalletRegistry). *The WalletRegistry is a total function $\text{WalletRegistry} : \mathcal{W} \rightarrow \text{WalletMetadata}$, where WalletMetadata contains non-economic fields: Know-Your-Customer status, permissions, the audit cursor, and operational flags. It is not part of the state layer.*

The WalletRegistry participates in authorisation and operational concerns; it does not participate in conservation, in PnL, or in time travel of economic state. Its separation from the three state stores is what makes Condition 4.11 easy to enforce by convention alone: if a field seems to want to live on WalletMetadata but carries economic content, it has been mis-placed and should live on a PositionState row keyed by the appropriate contract unit.

4.2 On Authorisation

Cross-wallet read authorisation—who may read PositionState under which wallet—is not addressed by the eleven conditions above. The v2 draft of this addendum included a “capability” condition at this point; it is withdrawn. The v10.3 main document does not define a capability layer, and introducing one here as a new primitive would exceed the addendum’s scope. Cross-wallet authorisation is the subject of a separate authorisation-layer specification. Readers of the eleven conditions should assume that per-wallet reads are authorised elsewhere.

5 How v10.3 §7 Is Refined

5.1 Replacement Text for Line 1034

The sentence at v10.3 line 1034 is replaced by the following block:

Each registered unit carries immutable, versioned `ProductTerms` and mutable, holder-shared `UnitStatus`. Each held position (w, u) carries a `PositionState` record, accessed through an `Option`-valued accessor in which `None` means “never held” and `Some` of a zero record means “held and currently flat”. State pertaining to a client’s relationship to a strategy, mandate, or managed account lives at `PositionState` under a contract unit—a mandate, CSA, or subscription agreement registered per v10.3 §6 and classified as a contract unit per Definition 2.5. There is no separate wallet-keyed state store; the `WalletRegistry` holds only operational metadata.

5.2 Replacement Signature for Line 2287

The accessor `view.get_unit_state(unit)` is refined into three accessors:

```
view.product_terms(unit)      -> NonEmptyList[TermsVersion]
view.unit_status(unit)        -> UnitStatusRecord
view.position_state(wallet, unit) -> Option[PositionStateRecord]
```

During migration, `get_unit_state(unit)` is retained as a deprecated alias that concatenates `product_terms(unit)` and `unit_status(unit)`. The single-argument form never returned per-holder data and so carries no migration risk; the two-argument form at v10.3 line 2287 is the dangerous one and is explicitly replaced by `position_state(wallet, unit)`.

5.3 What Else Moves

A handful of small refinements accompany the line 1034 replacement.

- The `clearinghouse` field at v10.3 line 1168 is re-described: when the same contract class can be cleared through multiple CCPs, each CCP gives rise to a distinct unit in `ProductTerms`; the per- (w, u) CCP binding in `PositionState` is an explicit acknowledgement of that partition, not a per-wallet override of a shared field.¹
- The `first_touch_date` field, which appears in some informal discussions of v10.3, is explicitly *not* a `PositionState` field. It is derived from the event log on demand. Caching it in `PositionState` would create a fold inconsistency under back-dated corrections.
- The “wallet-level smart contract” language of v10.3 §6 is preserved. The refinement makes explicit that such a contract’s state lives at `PositionState` under a contract unit rather than under a separate wallet-keyed store.

¹This modifies v10.3 §3.2’s unit identity rule for listed derivatives by including the clearing venue in the unit identity. The change is motivated by the non-fungibility of positions cleared at different CCPs for a given holder.

6 Four Test Cases

The original motivation for this refinement was a set of four concrete scenarios that v10.3 §7 handled but only at the cost of the binary per-unit/per- (w, u) split. This section shows how each scenario is handled under the three-store decomposition, with numeric traces.

6.1 A Future Whose State Depends on Past Transactions

Three clients take positions in the CME-cleared E-mini S&P futures contract: client *A* is long ten, client *B* is short fifteen, client *C* is long five. All three share the contract specification. Their accumulated costs, entry prices, and variation margin trajectories differ.

Table 2 records the store assignment.

Table 2: E-mini S&P: field placement.

Field	Store	Why it lives there
multiplier, currency, expiry, clearinghouse, exchange, product identifier	ProductTerms	Immutable contract specification. CME-cleared and ICE-cleared E-mini are distinct units.
lifecycle stage, last settlement price, last settlement date	UnitStatus	Shared observables. One settlement price per contract, published by the exchange and applied uniformly.
accumulated cost	PositionState	Per-holder by necessity: three clients, three different accumulated costs.

Numeric trace. Write *ac* for *accumulated_cost*, multiplier \$50/pt absorbed.

```

t=0 Trade: A buys 10 ES at 4500 from B
    PS[A, ES].ac -= 45000 ; PS[B, ES].ac += 45000
    row sum = 0 [C2]
t=1 Trade: C buys 5 ES at 4520 from B
    PS[C, ES].ac -= 22600 ; PS[B, ES].ac += 22600
    row sum = 0 [C2]
t=2 SettleVM(ES, close = 4510)
    PS[A, ES].ac += 100 (A gains 10 ticks at $1/tick)
    PS[B, ES].ac -= 50 (B, net short 5, loses 10 ticks)
    PS[C, ES].ac -= 50 (C, long 5, loses 10 ticks from 4520)
    row sum = 0 [C2]
t=3 Close: A sells 10 ES at 4510 to B
    PS[A, ES].ac += 45100 ; PS[B, ES].ac -= 45100
    row sum = 0 [C2]
    PS[A, ES] retained with ac = 100 (realised PnL) [C1 extensive]
```

After $t=3$, the PositionState row $[A, ES]$ is retained with *ac* equal to the realised gain. Tax reporting, wash-sale lookback, and year-end reconciliation all require the row to remain accessible. The Option accessor now returns *Some* of a non-zero record.

6.2 A Managed Account: The Inherently Wallet-Attached Case

A manager runs a discretionary portfolio on behalf of a client. The mandate specifies a fee schedule with a management fee and a performance fee subject to a high-water mark. The client's accrued fees, HWM, benchmark reference level, and breach flags appear to be per-wallet facts; they are specific to this client's relationship with this manager.

Section 6 of v10.3 admits mandates as “wallet-level smart contracts”. Here the smart contract is given its full identity as a *contract unit* per Definition 2.5. The manager issues it with balance -1 ; the client holds it with balance $+1$:

$$w_{\text{manager}}(u_{\text{MA}}) = -1, \quad w_{\text{client}}(u_{\text{MA}}) = +1, \quad \sum_w w(u_{\text{MA}}) = 0.$$

This is *bookkeeping conservation* (Remark 2.7): the manager does not owe the client a mandate back but owes performance under the mandate. The -1 names an accountable party, which is what distinguishes the scheme from the sentinel-unit scheme of Section 7.

Table 3 shows the field assignment under the contract unit.

Table 3: Mandate state, re-keyed from per-wallet to $(w_{\text{client}}, u_{\text{MA}})$.

Field	Store
Mandate text, fee schedule, benchmark identity, maximum position limits	ProductTerms[u_{MA}]
HWM hurdle methodology, crystallisation frequency	ProductTerms[u_{MA}]
HWM value (client-specific), HWM date	PositionState[$w_{\text{client}}, u_{\text{MA}}$]
Accrued management fee, accrued performance fee	PositionState[$w_{\text{client}}, u_{\text{MA}}$]
Mandate breach flags, subscription and redemption cursor	PositionState[$w_{\text{client}}, u_{\text{MA}}$]
Benchmark reference level at this client’s inception	PositionState[$w_{\text{client}}, u_{\text{MA}}$]
Current benchmark level (shared, from the index administrator)	UnitStatus[u_{bench}]

Two wallets, two mandates, no collapse. A client with a base discretionary mandate and a concurrent overlay mandate on the same wallet carries two distinct PositionState rows: one under $u_{\text{MA,base}}$ and one under $u_{\text{MA,overlay}}$, each with its own HWM, accrued fees, and breach flags. A flat per-wallet scalar representation would have collapsed them. This is the failure Condition 4.11 prevents.

6.3 A Quantitative Investment Strategy That Trades Futures

A Quantitative Investment Strategy is a rule-based tradable product whose return is generated by a systematic trading algorithm running against a declared universe. A client subscribes to the strategy; the strategy itself holds a trading wallet that executes the underlying trades. Five keyings coexist:

```

u_QIS -- the strategy unit (asset unit), tradable by clients
u_ES, u_NQ, u_YM -- futures the strategy holds in its trading wallet
u_MA_C -- a mandate contract unit, if the client holds QIS in a wrapper
w_QIS -- the strategy’s own trading wallet
w_C -- the client’s wallet

```

No two of these live in the same store. Table 4 shows the assignment.

Table 4: QIS state, split across the three stores by key.

Field	Store
-------	-------

Strategy contract terms: volatility target, barrier level, tradable universe, share-class index start level	ProductTerms[u_{QIS}]
Strategy current state: last rebalance date, current strategy weights, cumulative return, NAV index, realised volatility, triggered-barrier flag	UnitStatus[u_{QIS}] (strategy-level, shared across all holders)
Futures positions held by the strategy: accumulated cost, CCP binding	PositionState[w_{QIS}, u_{ES}], etc.
Per-client subscription state: entry NAV, HWM	PositionState[w_C, u_{QIS}]
Per-client mandate overlay, if present	PositionState[$w_C, u_{MA,C}$]

Rebalance row-set. A rebalance is not one event but a *compound*: one weight-update on UnitStatus[u_{QIS}] plus N trade events, one per underlying, whose counterparty rows are the executing CCP or market-maker. Conservation over `accumulated_cost` holds row-sum-zero on each trade; the UnitStatus weight update is not a conserved-field write. The atomic StateDelta spans three stores and $N + 1$ counterparty wallets.

Numeric trace. A weekly rebalance moves weights (ES, NQ, YM) from (0.50, 0.30, 0.20) to (0.40, 0.35, 0.25). Strategy NAV is \$10,000,000; contract multipliers 50/20/5; prices ES 4500, NQ 15000, YM 35000. Executing CCP is w_{CME} .

```

t=0  UpdateWeights(u_QIS, (0.50,0.30,0.20) -> (0.40,0.35,0.25))
      US[u_QIS].weights := (0.40, 0.35, 0.25)
      (not a conserved-field write)                                     [C3]

t=0  Trade: w_QIS sells 4 ES at 4500 to w_CME
      PS[w_QIS, ES].ac += 900000 ; PS[w_CME, ES].ac -= 900000
      row sum = 0                                                       [C2]

t=0  Trade: w_QIS buys 2 NQ at 15000 from w_CME
      PS[w_QIS, NQ].ac -= 600000 ; PS[w_CME, NQ].ac += 600000
      row sum = 0                                                       [C2]

t=0  Trade: w_QIS buys 3 YM at 35000 from w_CME
      PS[w_QIS, YM].ac -= 525000 ; PS[w_CME, YM].ac += 525000
      row sum = 0                                                       [C2]

```

The four lines above are one atomic StateDelta (C3). Without atomicity, a reader observing UnitStatus might see the new weights before the corresponding futures positions appear, producing a transient inconsistency visible to every client.

HWM placement. A client with two strategy subscriptions must carry two HWMs. The only representation that composes correctly is the per- (w, u_{QIS}) row. When the client holds the strategy inside a mandate wrapper, the HWM of record for performance-fee purposes is on $(w_C, u_{MA,C})$; when no wrapper mandate is in force, it is on (w_C, u_{QIS}) directly. The distinction is carried by the mandate ProductTerms, not by a convention on the HWM value.

Wind-down. Transitioning the strategy's lifecycle stage to CLOSED propagates to clients as a NAV-strike redemption event. Each client's PositionState row is updated to a zero balance and *retained* by the extensive carrier discipline. The retained rows preserve the audit trail, the final HWM, and the per-client performance history for tax reporting.

6.4 An Instrument Listed in the Universe but Not Yet Traded

A listed option series is created on listing day, long before any holder takes a position. The framework must represent this state cleanly: the unit exists, its terms are fixed, its lifecycle stage is LISTED, and no wallet holds it.

- ProductTerms: present, with the initial TermsVersion written at registration (Condition 4.6).
- UnitStatus: present and fully initialised with product-declared defaults—for an option, lifecycle_stage is LISTED, last_settlement_price is absent, triggered_barrier is false (Condition 4.4). The accessor unit_status is total.
- PositionState: no rows for any wallet. The accessor position_state returns None for every wallet until first touch (Condition 4.1).
- WalletRegistry: unaffected.

Lifecycle totality. The sequence LISTED → ACTIVE → EXPIRED occurs entirely within Unit-Status, producing zero PositionState writes. Each transition fires with an empty holder set. The vacuous base case discharges the conservation obligation: the sum over zero rows is zero.

The untraded-instrument bug. Condition 4.8 catches a concrete bug class. The following dividend handler crashes on a listed-but-untraded security:

```
# Naive: crashes on listed-but-untraded unit
def dividend_handler(u, total):
    holders = all_holders_of(u)
    per = total / len(holders)    # ZeroDivisionError
    return [Move(treasury, w, per, cash) for w in holders]
```

The fix is to discharge conservation vacuously:

```
# Correct: vacuous discharge of conservation
def dividend_handler(u, total):
    holders = all_holders_of(u)
    if not holders:
        return []                # sum = 0 trivially (C9)
    per = total / len(holders)
    return [Move(treasury, w, per, cash) for w in holders]
```

Amendment discipline. Amendments flow through Condition 4.7. A fungibility-preserving amendment—a coupon-step-up within the declared corridor, a CSA eligible-collateral extension, an administrative fee-schedule tweak—appends a new TermsVersion. Existing PositionState rows, if any, are untouched. A fungibility-breaking amendment—an ISIN change, a benchmark swap, a bond restructuring—allocates a fresh unit. The new terms are written as the new unit's ProductTerms, a supersession link is stamped into the old unit's UnitStatus, and an atomic re-subscription StateDelta re-keys each position from the old unit to the new while preserving the conservation sum.

7 Alternatives Considered

Three alternative representations were considered before the three-store decomposition was adopted. Each fails for a specific reason, and the nature of the failure is instructive: it shows that removing any of the three stores, or adding a fourth, is forced by a corresponding feature of the problem.

7.1 Two Stores (ProductTerms and PositionState Only)

A seemingly simpler scheme fuses UnitStatus into PositionState by introducing a “universe wallet” w_* that holds the shared observables: the last settlement price sits on $[w_*, u]$, the strategy weights on $[w_*, u_{QIS}]$, and so on. The scheme promises to reduce the number of stores from three to two.

It fails at conservation. The conservation law must now be re-stated to exclude w_* , and every conserved-field handler must be taught to exclude it. The per- u vs. per- (w, u) distinction that the scheme was supposed to eliminate has re-appeared as a runtime predicate on the wallet axis. Worse, the distinction is now enforced by convention (a wallet identifier check) rather than by the type system.

7.2 Four Stores (Keeping an Explicit WalletState)

A four-store scheme retains an explicit WalletState store keyed by w alone, containing per-wallet economic fields: a wallet-keyed HWM, a wallet-keyed accrued fee, a wallet-keyed benchmark reference level. Every such field must be assigned to some mandate or strategy to be meaningful—a client’s HWM is a HWM *under some mandate*; there is no such thing as a mandate-free HWM.

Enumerating the fields a WalletState would contain, each turns out to be naturally keyed by a contract unit. The HWM, accrued fees, and breach flags live at (w, u_{MA}) . The benchmark reference level lives at (w, u_{MA}) . The subscription and redemption cursor lives at (w, u_{MA}) (or (w, u_{QIS}) when the client holds the strategy without a wrapper mandate). The putative WalletState is empty of economic content. The four-store scheme introduces a sector whose cardinality is zero.

7.3 A Sentinel-Unit Scheme (“Dirac”)

A further scheme introduces a sentinel unit $u_\emptyset$ under which per-wallet state is keyed as $\text{PositionState}[w, u_\emptyset]$. The scheme is internally consistent except at the conservation law: $u_\emptyset$ has no issuer, no identifiable holder on the other side, and no accountable party standing behind the -1 balance. The conservation sum $\sum_w w(u_\emptyset) = 0$ would have to be declared by fiat rather than earned by an accountability discipline. Contrast the contract unit of Definition 2.5: its -1 balance is carried by the manager, who is a legal person accountable for performance. See Remark 2.7. A system that carves out exceptions to its conservation law without naming the party behind them pays an ongoing correctness tax; the scheme is rejected on this ground.

7.4 Why Exactly Three

Three independent forcing constraints each break a different store:

1. *Two wallets holding the same contract must be distinguishable.* Distinct accumulated costs, distinct entry NAVs, distinct HWMs require $\text{PositionState}[w, u]$. Removing it is impossible.
2. *Shared observables exist.* A settlement price, an index level, a strategy weight vector is one-per-contract and is dereferenced by every holder; duplicating it across holders is both wasteful and inconsistent, so a u -keyed store distinct from PositionState is required.

3. *The audit discipline of terms differs from the mutation rate of status.* Versioned append-only amendment history conflicts with per-settlement mutation; merging them under a single name forces a compromise that breaks one of the two use cases.

The three stores are pinned by points (1), (2), and (3) respectively. Removing any of the three destroys the corresponding invariant. Adding a fourth introduces a sector whose economic cardinality is zero. Three is the minimum faithful decomposition; it is not a compromise.

8 Structurally Unreachable Invariants

“Structurally unreachable” is defined in Definition 2.15: a state predicate is structurally unreachable in the persistent ledger state when no sequence of well-typed events, applied through the `apply` boundary with conservation enforcement, can yield a committed state satisfying it. This section establishes that the *negation* of each of two v10.3 §11 invariants, and of three additional invariants internal to this addendum, is structurally unreachable.

Restricting attention to the core ten v10.3 §11 invariants (P1–P10; SBL-specific invariants P11–P20 and obligation-liveness invariants P21–P23, declared elsewhere in v10.3, are outside this addendum’s scope): P1 is conservation; P8 is snapshot consistency (time travel). The other eight—P2 atomic commitment, P3 referential integrity, P4 log monotonicity, P5 transaction idempotency, P6 lifecycle idempotency, P7 virtual/real isolation, P9 lifecycle purity, P10 PnL path-independence—are not newly made structurally unreachable by this addendum; they are already enforced by v10.3’s existing primitives (atomic transactions, append-only move stream, deduplication by `tx_id`, virtual wallet taxonomy, state-sufficiency of valuation).

This addendum introduces three new invariants, labelled Q1 through Q3, that are specific to the three-store decomposition and that are not in the v10.3 §11 list. They are: Q1 immutability of terms, Q2 permanence of unit identity, and Q3 handler-field canon. The labels are fresh to make clear that these are addendum-level commitments.

Honest count: two v10.3 invariants whose negation is structurally unreachable (P1, P8), plus three addendum-level invariants (Q1, Q2, Q3). Five in total.

8.1 v10.3 Invariant P1: Conservation

Theorem 8.1 (C1 ensures non-violation of P1). *The negation of v10.3 Invariant P1—the existence at some time t and some asset unit u of a committed ledger state in which $\sum_w w_t(u) \neq 0$ —is structurally unreachable (Definition 2.15).*

Sketch. Each event handler produces a `StateDelta` satisfying handler-level conservation for each conserved field (Condition 4.2), applied atomically (Condition 4.3). `Ledger.apply` (see §11) computes the per-field row sum and raises on non-zero sum; a non-conserving delta does not reach persistence. Induction on the event stream yields the global invariant. The vacuous base case (Condition 4.8) covers events on zero-holder units. $\square$

Remark 8.2 (Rejection vs. prevention). *Conservation is enforced by synchronous rejection at the `apply` boundary. The illegal state is never persisted, but it is constructible in handler code up to the `apply` call. A stronger “compile-time structural unreachability” would require refinement types on handler output and is a future direction. In the sense of Definition 2.15—unreachable in the persistent state—conservation is structurally enforced; in a stronger sense, it is runtime-enforced.*

For contract units, the conservation sum holds by the bookkeeping discipline of Remark 2.7 rather than by market mechanism, but the theorem applies in the form stated.

8.2 v10.3 Invariant P8: Snapshot Consistency

Theorem 8.3 (C1 ensures non-violation of P8). *The negation of v10.3 Invariant P8—the existence of an event stream E and an index k for which $\text{apply_all}(E) \neq \text{apply_from}(\text{apply_all}(E[0:k]), E[k:])$ in the persistent state—is structurally unreachable.*

Sketch. Three hypotheses are required for replay-to-snapshot equivalence:

1. Events are totally ordered under a reproducible ordering (v10.3 P4 log monotonicity and the sequencer’s timestamp/tiebreak discipline).
2. Handlers are pure of hidden inputs (v10.3 §7.5 purity).

3. `apply(delta, state)` is deterministic on its inputs (v10.3 §2.3 move semantics).

Condition 4.1, carrier clause (extensive key-sets, Definition 2.9), adds the fourth hypothesis needed to lift replay-to-snapshot equivalence *through close-outs*: no event’s behaviour depends on the *absence* of a key. Under (1)–(3) from v10.3 and the extensive-key-set discipline from this addendum, Remark 2.16 yields the fold identity, which is P8 in the notation of Definition 2.14. $\square$

8.3 Addendum Invariant Q1: Terms Immutability

Invariant 8.4 (Q1: Terms immutability). *An already-written TermsVersion is never mutated. Amendment appends a new version; breaking amendment allocates a fresh unit.*

Theorem 8.5 (C6 and C7 ensure non-violation of Q1). *The negation of Q1 is structurally unreachable.*

Sketch. By Condition 4.5 the TermsVersion list is an append-only `NonEmptyList` with no mutation API. By Condition 4.7 amendments route through one of two named paths, neither of which mutates an existing version. $\square$

8.4 Addendum Invariant Q2: Identity Permanence

Invariant 8.6 (Q2: Unit identity permanence). *A unit identifier once registered is never re-assigned. Breaking amendments allocate a fresh identifier; the old is preserved under a supersession link.*

Theorem 8.7 (C9 and C7 ensure non-violation of Q2). *The negation of Q2 is structurally unreachable.*

Sketch. Re-registration is rejected by Condition 4.9. Breaking amendments, by Condition 4.7, allocate a fresh identifier and stamp a supersession link on the old unit’s UnitStatus; neither path reassigns an existing identifier. $\square$

8.5 Addendum Invariant Q3: Handler-Field Canon

Invariant 8.8 (Q3: Handler-field canon). *Every PositionState field has a unique canonical writer.*

Theorem 8.9 (C10 ensures non-violation of Q3). *The negation of Q3 is structurally unreachable.*

Sketch. By Condition 4.10, which is a type-level gate on field access. Two handler classes attempting to write the same field fail to type-check. $\square$

8.6 What the Addendum Does Not Make Structurally Unreachable

Six of the v10.3 §11 invariants are not re-established here because they are already enforced by v10.3’s primitives, independently of the state decomposition: P2 atomic commitment (transaction boundary), P3 referential integrity (wallet/unit existence check), P4 log monotonicity (append-only move stream), P5 transaction idempotency (`tx_id` deduplication), P7 virtual/real isolation (wallet taxonomy), P9 lifecycle purity (pure handler functions), P10 PnL path-independence (state-sufficient valuation). P6 lifecycle idempotency remains a handler obligation and is assisted by Q3 (handler-field canon gives idempotency a single-owner key), but is not itself structurally unreachable in persistent state; an idempotent handler is a design discipline, not a type-level guarantee in the sense of Definition 2.15.

9 Testing Targets

The three-store decomposition constrains the test programme. Because the stores have different mutation disciplines, they have different mutation-testing profiles. The following figures are targets, not commitments.

- **Event handlers over conserved Decimal fields.** Target mutation score: 85–90 percent. The handler-level conservation constraint kills most sign and coefficient mutants directly—a mutated delta that no longer sums to zero fails the conservation test immediately.
- **Lifecycle guards.** Target mutation score: 70–80 percent. Boundary-comparison mutants—replacing `>` with `>=`, or `<=` with `<`—require targeted tests around the lifecycle transition points; they are not caught by conservation alone.
- **Core state layer overall.** Target mutation score: at least 80 percent. This meets the common industry threshold for change-safe core code.
- **Reachable state exploration.** For small state-machine configurations—three wallets, two units, event-sequence depth at most six—the reachable state space is of order 10^5 to 10^6 configurations, tractable for an explicit-state model checker. A conservation-violating handler is caught at depth one.

The generator universe for property-based testing is the Cartesian product of CDM enumerations (v10.3 §10, §11) with the product-type universe. Because both components are finite and enumerable, test coverage is structurally bounded.

10 Risk Register for Migration

The refinement does not invalidate any correctness property of v10.3. It does introduce migration, governance, and operational risks that a programme plan must budget for. The eight risks below have been reviewed and none is a correctness blocker; each is an engineering or stakeholder concern.

Table 5: Migration risk register. Items marked with an asterisk require stakeholders outside the engineering team before implementation can begin.

#	Risk	Mitigation
R1	Migration scope: every downstream reader of the legacy <code>get_unit_state</code> accessor must be updated.	Retain <code>get_unit_state</code> as a deprecated alias that concatenates <code>product_terms</code> and <code>unit_status</code> ; land the additive split before the cutover; run the parallel representations for at least one quarter and reconcile.
R2*	Governance of the fungibility predicate (Condition 4.7): who declares, who reviews, who retracts.	Assign a per-product RACI in the Unit Store registry; require Legal, Product, and Risk sign-off for any predicate change; version the predicate itself alongside <code>TermsVersion</code> .
R3	Extensive carrier growth at the target scale of roughly 10^7 wallets and 10^6 units is unbenchmarked.	Benchmark the Option-accessor cost and cache behaviour before merge. Accept unbounded key-set growth as a genuine operational cost of the extensive-carrier discipline; a tombstone-with-compaction policy would contradict Condition 4.1 (closed-and-compacted would be indistinguishable from never-held) and is not adopted. Focus R3 mitigation on capacity planning and retrieval-path caching.
R4	The full set of Conditions 4.1–4.11 is a multi-sprint test programme, not a single release.	Land the schema first with Conditions 4.1, 4.4, 4.5, 4.6, 4.8, and 4.9—all type- or schema-enforced and available at no marginal test cost. Stage Conditions 4.2, 4.3, 4.7, and 4.10 over subsequent releases. Never merge without at least Conditions 4.1, 4.2, and 4.3.
R5*	Mandate-as-contract-unit may expand the regulatory reporting surface for mandate issuance under SFTR and EMIR.	Pre-flight with Regulatory before implementation; determine whether mandate-unit issuance requires a UTI and an LEI pair; provide a <code>reportable</code> flag on the mandate’s Product-Terms to opt an issuance out of reporting where the regulation permits.
R6*	CDM alignment of <code>PositionState</code> against the CDM <code>TradeState</code> shape is claimed but not re-verified under the three-store decomposition.	Re-run the Rosetta namespace mappings (NS1 through NS7) against the three-store schema before any CDM adapter work; publish a delta document describing any mapping changes.

R7	Onboarding cost: three stores, two orthogonal disciplines on Position-State, and eleven conditions is a non-trivial mental model.	Provide the decision tree of §2.7, a runbook, and a one-page cheat sheet. Mandatory training before new engineers touch handler code.
R8	The forward migration is not trivially reversible once legacy WalletState fields have been re-keyed.	Keep a read-only mirror of the legacy representation for at least four quarters post-cutover; document the inverse mapping (contract unit back to the former WalletState field) explicitly in the migration runbook.

R2, R5, and R6 each require engagement outside the engineering team before any code ships. R1, R3, R4, R7, and R8 are internal engineering risks.

11 Reference Implementation

The following listing is a minimal, runnable sketch of the state layer. It is not a production implementation; it is a demonstration that the design is concrete and that the conditions are mechanically checkable.

The `Ledger` class, presented first, carries every lesson: three stores (PT, US, PS), registration, position lookup, apply, amend. The supporting dataclasses (`NonEmpty`, `TermsVersion`, `UnitStatus`, `PositionState`) follow as backdrop.

```
from dataclasses import dataclass
from typing import Callable, Generic, Optional, TypeVar

U = TypeVar("U"); W = TypeVar("W"); T = TypeVar("T")

class Ledger:
    def __init__(self) -> None:
        self.PT: dict = {} # ProductTerms (Conditions 5, 6)
        self.US: dict = {} # UnitStatus (Condition 4)
        self.PS: dict = {} # PositionState (Condition 1)

    def register(self, u, tv, us) -> None:
        # Condition 9: re-registration rejected
        # Condition 4: UnitStatus total at registration
        # Condition 6: ProductTerms total at registration
        if u in self.PT:
            raise ValueError("C9: re-registration rejected")
        self.PT[u] = NonEmpty(tv)
        self.US[u] = us

    def position(self, w, u) -> Optional["PositionState"]:
        # Condition 1: Option accessor; no delete operator exposed
        return self.PS.get((w, u))

    def apply(self, delta: dict) -> None:
        # Condition 3: atomic three-store application
        # Condition 2: handler-level conservation per conserved field
        # Condition 10: handler-field canon (checked via FIELD_SPEC tokens)
        for f, spec in FIELD_SPEC.items():
            if spec["conserved"]:
                s = sum(d.get(f, 0) for d in delta["rows"].values())
                if s != 0:
                    raise ValueError(f"C2: {f} not conserved (sum={s})")
        # Begin atomic write across PT, US, PS (Condition 3)
        for u_key, us_update in delta.get("us", {}).items():
            self.US[u_key] = us_update
        for (w, u), diff in delta["rows"].items():
            old = self.PS.get((w, u), PositionState())
            # Extensive carrier: row is never deleted (Condition 1)
            self.PS[(w, u)] = PositionState(**{**old.__dict__, **diff})
        # End atomic write

    def amend(self, u, tv_new, fresh: Callable[[], object]):
        # Condition 7: two-track amendment, fungibility-predicate-driven
        head = self.PT[u].current()
        if head.is_fungibility_preserving(tv_new):
            self.PT[u] = self.PT[u].append(tv_new) # preserving track
            return u
        # breaking track: fresh unit, supersession link
```

```

    u_new = fresh()
    self.PT[u_new] = NonEmpty(tv_new)
    self.US[u] = UnitStatus(
        **{**self.US[u].__dict__, "superseded_by": u_new})
    return u_new

# Supporting dataclasses follow.

@dataclass(frozen=True)
class NonEmpty(Generic[T]):
    head: T
    tail: tuple = ()
    def append(self, x): # Condition 5: append-only API
        return NonEmpty(self.head, self.tail + (x,))
    def current(self):
        return self.tail[-1] if self.tail else self.head

@dataclass(frozen=True)
class TermsVersion:
    fields: dict
    is_fungibility_preserving: Callable[["TermsVersion"], bool]
    # Fungibility predicate is product-declared (Definition 2.16):
    # deterministic, monotone under field change, conservation-safe.

@dataclass(frozen=True)
class UnitStatus:
    lifecycle: str
    last_px: Optional[float]
    superseded_by: Optional[object] = None

@dataclass(frozen=True)
class PositionState:
    accumulated_cost: float = 0.0
    balance: float = 0.0
    hwm: float = 0.0

# Condition 10: each field carries its canonical handler tag and
# its conservation status (set at registration per Definition 2.10).
FIELD_SPEC = {
    "accumulated_cost": {"conserved": True, "handler": "settle"},
    "balance": {"conserved": True, "handler": "transfer"},
    "hwm": {"conserved": False, "handler": "fee_crystallise"},
}

```

The listing exhibits the Option accessor (`position` returns `Optional[PositionState]`), the extensive carrier (no delete operator), the atomic three-store `StateDelta` (`apply` either applies the full delta or raises), the handler-level conservation (checked per field), the re-registration rejection (Condition 4.9), the two-track amendment (Condition 4.7), and the field-canon tagging (Condition 4.10 through `FIELD_SPEC`). The fungibility predicate is declared on `TermsVersion`, satisfying its product-declared origin.

12 Summary

The refinement changes the phrasing at v10.3 line 1034 and the accessor signature at v10.3 line 2287. Nothing else in v10.3 changes.

Section 7 of v10.3 correctly observed that unit state for some instruments must be keyed by (w, u) and for others by u alone. This addendum refines the observation into a three-store decomposition—ProductTerms, UnitStatus, PositionState—plus a partition of the unit universe into asset units and contract units. Eleven conditions govern the use of the stores. The putative fourth sector (a wallet-keyed state store) collapses into PositionState keyed by a contract unit.

Under the refinement, the negation of two v10.3 §11 invariants (P1 conservation, P8 snapshot consistency) is structurally unreachable in the persistent state, and three additional invariants internal to this addendum (Q1 terms immutability, Q2 identity permanence, Q3 handler-field canon) are likewise structurally unreachable. The remaining eight v10.3 invariants are enforced by v10.3’s existing primitives, not by the state decomposition. The four test cases—the future, the managed account, the QIS strategy, and the listed-but-untraded instrument—exhibit the decomposition with numeric traces. The risk register lists the concrete migration and governance risks.